

# Neural Network Basics in Sports Statistics

Vinnie Carbone

## Neural Networks in Sports Analytics

### What is a Neural Network?

- A neural network is a statistically trained model that draws inspiration from the human brain
- It learns from data in order to make predictions
- It is not explicitly programmed, the key here is that it **learns**

### Why Use Neural Networks?

- Able to see **complex relationships** in data
  - Works best with nonlinear patterns
- Used commonly in many fields including:
  - Image recognition
  - Speech recognition
  - Sports analytics

### Today's Goal

- Build a simple neural network to predict NBA player performance. We will predict:
  - Points, rebounds, and assists.
- Compare predicted vs actual stats to see how well the neural network predicts

# How Neural Networks Work

## Basic Structure

- Neural networks are made up of layers of neurons (the human brain has billions of neurons)
  - Input layer
  - Hidden layers
  - Output layer
- Each neuron (or node):
  - Takes in numbers
  - Applies a transformation
  - Passes the result forward

## Flow of Information

- Data moves **forward** through the network
- Each layer transforms the data slightly
- Final layer produces a prediction

## Simple Diagram

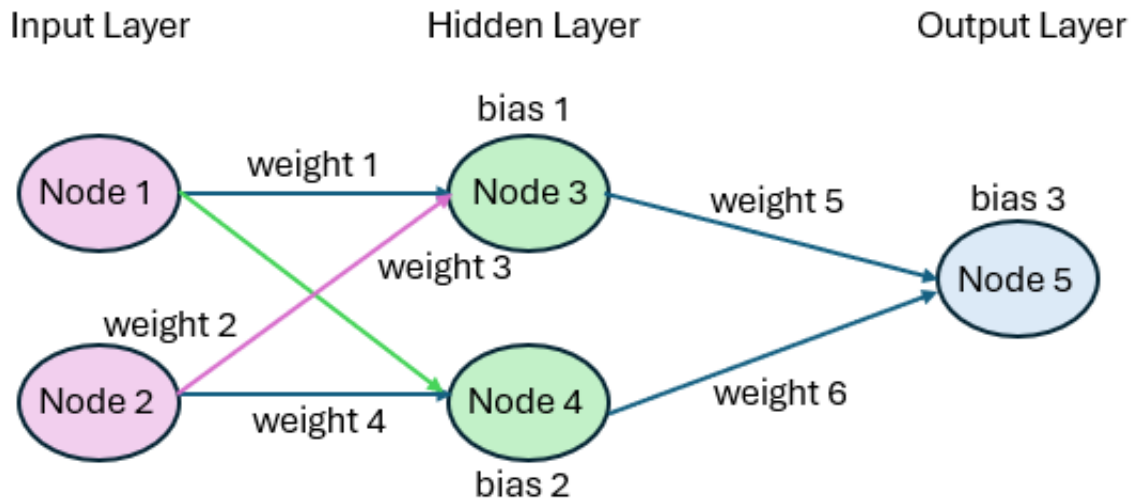


Figure 1: Neural network diagram

## What Happens Inside a Neural Network?

### Neurons (Nodes)

- Each circle in the network is called a **neuron/node**
- A neuron:
  - Receives inputs
  - Processes them
  - Passes an output forward

### Weights and Bias

- **Weights:**
  - Control how important each input is
  - Larger weight = more influence on the prediction
- **Bias:**
  - A constant added to help shift predictions
  - Allows the model to be more flexible

## Activation Function (ReLU)

- After combining inputs, neurons apply an **activation function**
- We used **ReLU (Rectified Linear Unit)**:

$$f(x) = \max(0, x)$$

- This means:
  - If input is negative  $\rightarrow$  output is 0
  - If input is positive  $\rightarrow$  keep it

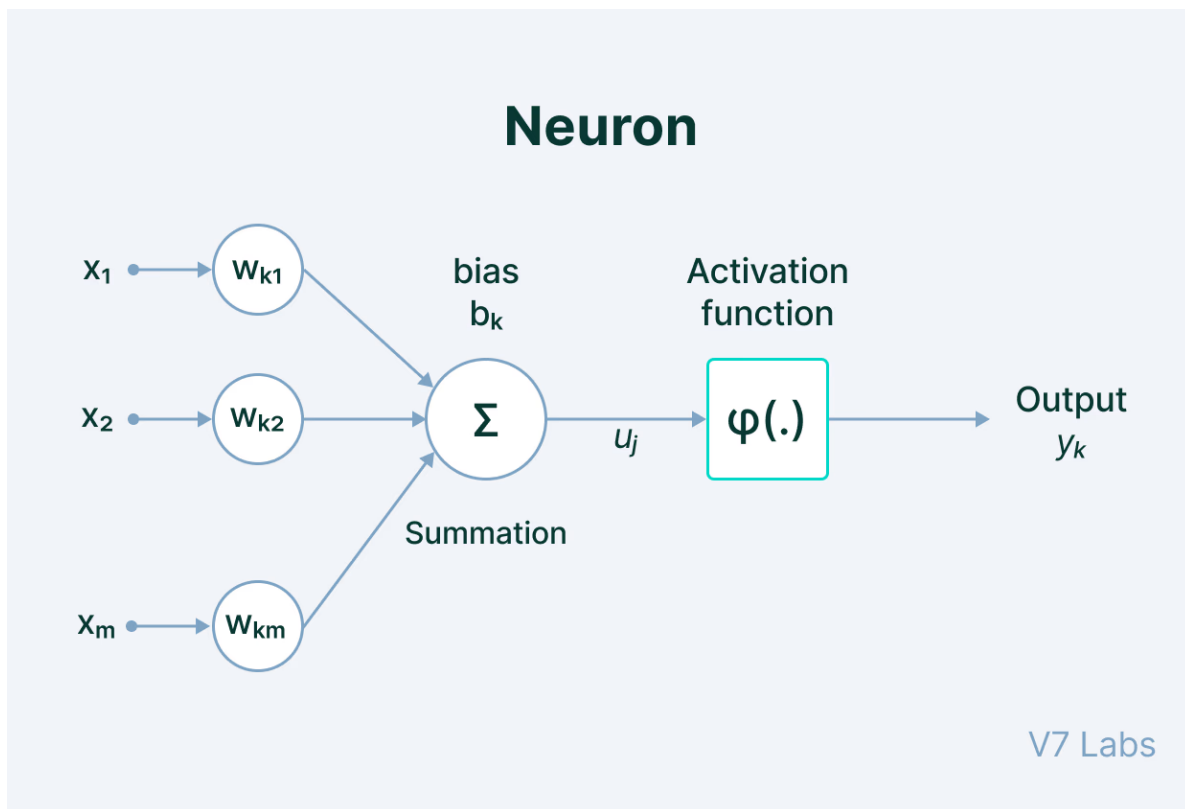


Figure 2: Neuron diagram

## Why This Matters

- ReLU helps the model learn **nonlinear patterns**
- This is important because:
  - Player performance is not perfectly linear

## Other Activation Functions

### Sigmoid

- Outputs values between **0 and 1**
- Often used for probabilities

$$f(x) = \frac{1}{1 + e^{-x}}$$

- Limitation:
  - Can slow down learning in deeper networks

### Tanh (Hyperbolic Tangent)

- Outputs values between **-1 and 1**
- Centered at 0 (helps learning compared to sigmoid)

$$f(x) = \tanh(x)$$

- Still can have issues with slow learning in deep networks

### Key Takeaway

- Different activation functions change how the model learns patterns
- ReLU is most common, but others are useful in specific situations

## Example of a Neural Network in Sports Statistics

### Data

- Using Basketball-Reference.com, I imported these eight stats from players in the NBA since the 2019 season
- Data cleaning: 0 and NA are not the same thing in this context

Click to expand code

```
lebron_df = all_df[all_df["player"] == "LeBron James"] \
    .sort_values("season")[
        ["player", "season", "age", "mpg", "games", "fga", "efg", "pts", "trb", "ast",]
    ]
lebron_df
```

|      | player       | season | age | mpg  | games | fga  | efg   | pts  | trb | ast  |
|------|--------------|--------|-----|------|-------|------|-------|------|-----|------|
| 2217 | LeBron James | 2019   | 34  | 35.2 | 55    | 19.9 | 0.560 | 27.4 | 8.5 | 8.3  |
| 13   | LeBron James | 2020   | 35  | 34.6 | 67    | 19.4 | 0.550 | 25.3 | 7.8 | 10.2 |
| 545  | LeBron James | 2021   | 36  | 33.4 | 45    | 18.3 | 0.576 | 25.0 | 7.7 | 7.8  |
| 1609 | LeBron James | 2022   | 37  | 37.2 | 56    | 21.8 | 0.590 | 30.3 | 8.2 | 6.2  |
| 1077 | LeBron James | 2023   | 38  | 35.5 | 55    | 22.2 | 0.549 | 28.9 | 8.3 | 6.8  |
| 3297 | LeBron James | 2024   | 39  | 35.3 | 71    | 17.9 | 0.599 | 25.7 | 7.3 | 8.3  |

## Model Structure

- Input features:
  - Previous season stats: age, minutes per game (mpg), field goal attempts (fga), effective field goal percentage (efg), points (pts), rebounds (trb), assists (ast)
- Output:
  - Predicted points, rebounds, and assists for the next season

Click to expand code

```
lebron_df_2 = all_df[all_df["player"] == "LeBron James"] \
    .sort_values("season")[
        ["player", "season", "prev_age", "prev_mpg", "prev_games", "prev_fga", "prev_efg", "prev_pts", "prev_ast",]
    ]
lebron_df_2
```

|      | player       | season | prev_age | prev_mpg | prev_games | prev_fga | prev_efg | prev_pts | prev_ast |
|------|--------------|--------|----------|----------|------------|----------|----------|----------|----------|
| 2217 | LeBron James | 2019   | 33.0     | 36.9     | 82.0       | 19.3     | 0.590    | 27.5     | 8.6      |
| 13   | LeBron James | 2020   | 34.0     | 35.2     | 55.0       | 19.9     | 0.560    | 27.4     | 8.5      |
| 545  | LeBron James | 2021   | 35.0     | 34.6     | 67.0       | 19.4     | 0.550    | 25.3     | 7.8      |
| 1609 | LeBron James | 2022   | 36.0     | 33.4     | 45.0       | 18.3     | 0.576    | 25.0     | 7.7      |
| 1077 | LeBron James | 2023   | 37.0     | 37.2     | 56.0       | 21.8     | 0.590    | 30.3     | 8.2      |
| 3297 | LeBron James | 2024   | 38.0     | 35.5     | 55.0       | 22.2     | 0.549    | 28.9     | 8.3      |

## Inputs & Targets

- We started with stats from 2018 - 2024. We dropped the first year, because this year has no 'previous season'.

```
# Assigning previous season stats as features / inputs
X = all_df[[
    "prev_age",
    "prev_games",
    "prev_mpg",
    "prev_fga",
    "prev_efg",
    "prev_pts",
    "prev_trb",
    "prev_ast"
]]

# Targets: this season's points, rebounds and assists.
y = all_df[["pts", "trb", "ast"]]
```

## Architecture

- Input layer → 8 features
- Hidden Layer 1 → 64 neurons (ReLU)
- Hidden Layer 2 → 32 neurons (ReLU)
- Output Layer → 3 neurons (PTS, TRB, AST)

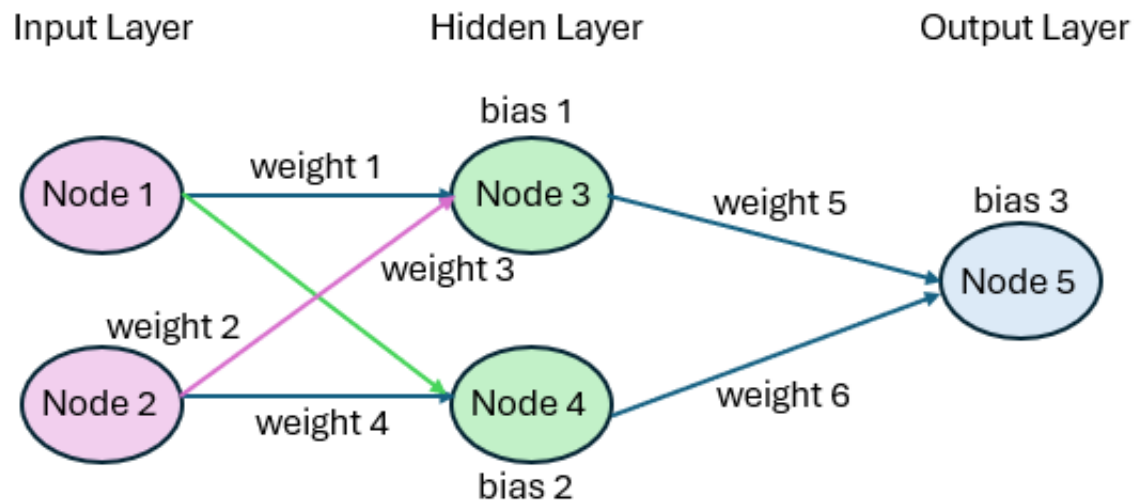


Figure 3: Neural network diagram

### Why 64 and 32?

- More neurons = more ability to learn patterns
- First layer (64):
  - Captures complex relationships in player stats
- Second layer (32):
  - Refines and simplifies those patterns

(insert code)

### Training the Model

- The model learns by adjusting:
  - Weights
  - Biases

Click to expand code



```
# Scaling
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

- Random training/testing split, rows are shuffled before splitting so the model sees different players and different seasons
- **random\_state=42** sets the random seed just saw the data splits the same time every run.

```
# Train/test split

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled,
    y,
    test_size = 0.2,
    random_state = 42
)
print(X_train.shape)
print(X_test.shape)
print(y_train.shape)
print(y_test.shape)
```

```
(2120, 8)
(531, 8)
(2120, 3)
(531, 3)
```

## Epochs

- Used TensorFlow library which has tools to create, train, and optimize neural networks
- We trained for 50 **epochs**
  - Each epoch = one full pass through the training data
  - Each epoch looks through the data 32 rows at a time (batch\_size)
- **Train Loss** = error on the data the model is learning from (learning)
- **Validation Loss** = error on the data the model has NOT trained on (generalization)

- Mean Squared Error (MSE/Train Loss) as the loss function to train the model, and Mean Absolute Error (MAE) as a metric to interpret how far off our predictions were

Click to expand code

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

# Define the model
model = keras.Sequential([
    layers.Dense(64, activation="relu", input_shape=(X_train.shape[1],)),
    layers.Dense(32, activation="relu"),
    layers.Dense(3) # 3 outputs: pts, trb, ast
])

# Compile the model
model.compile(
    optimizer="adam",
    loss="mse",      # mean squared error
    metrics=["mae"]  # mean absolute error
)

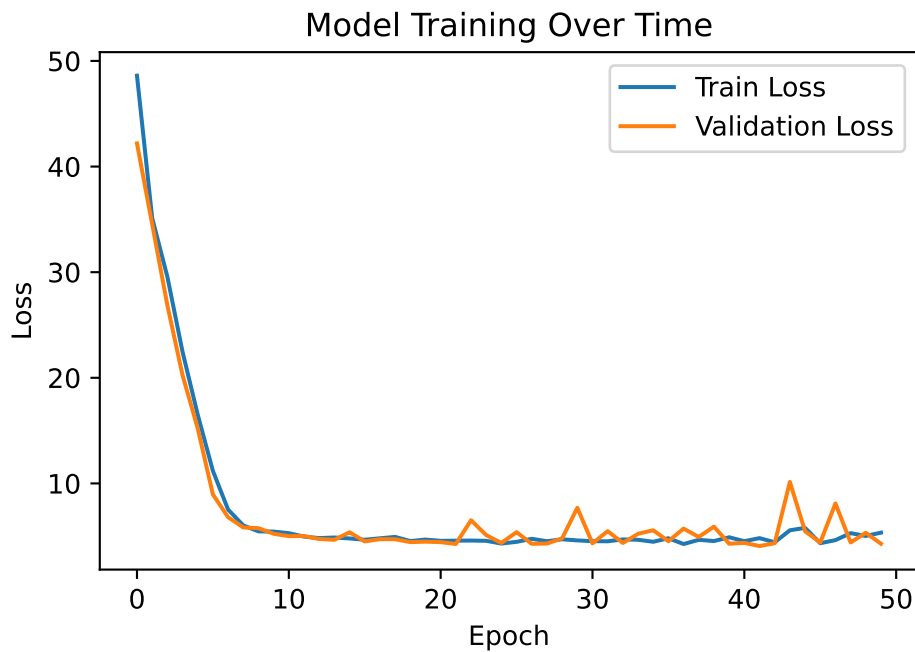
history = model.fit(
    X_train,
    y_train,
    epochs=50,
    batch_size=32,
    validation_split=0.2,
    verbose=0
)

import matplotlib.pyplot as plt

plt.plot(history.history["loss"], label="Train Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Model Training Over Time")
plt.legend()
plt.show()
```

/Users/vinniecarbone/Documents/3255/Presentation/tfenv/lib/python3.11/site-

```
packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```



## Key Idea

- The network learns patterns like:
  - “Higher minutes + higher usage → more points”
  - “Aging + lower minutes → possible decline”

## Results and Model Performance

### Overall Accuracy

- $MAE = [pts\ error + trb\ error + ast\ error] / 3$ 
  - The model is off by about 1.4 stats per game on average

Model MSE: 4.723

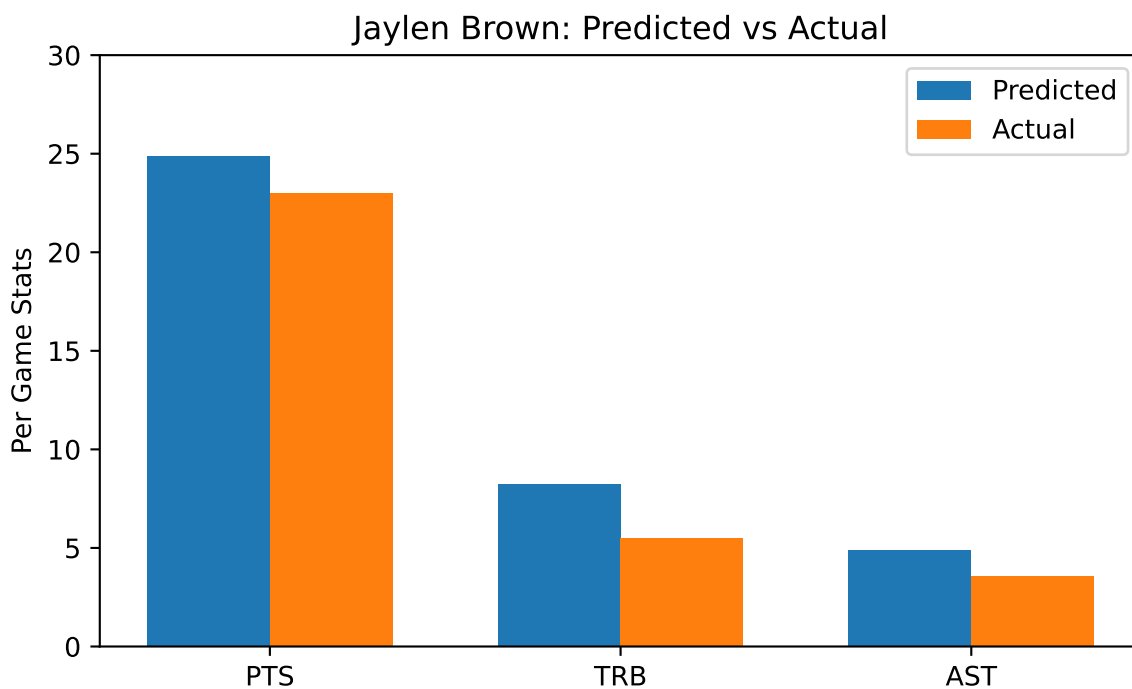
Model MAE: 1.432

Model RMSE: 2.1731829101289346



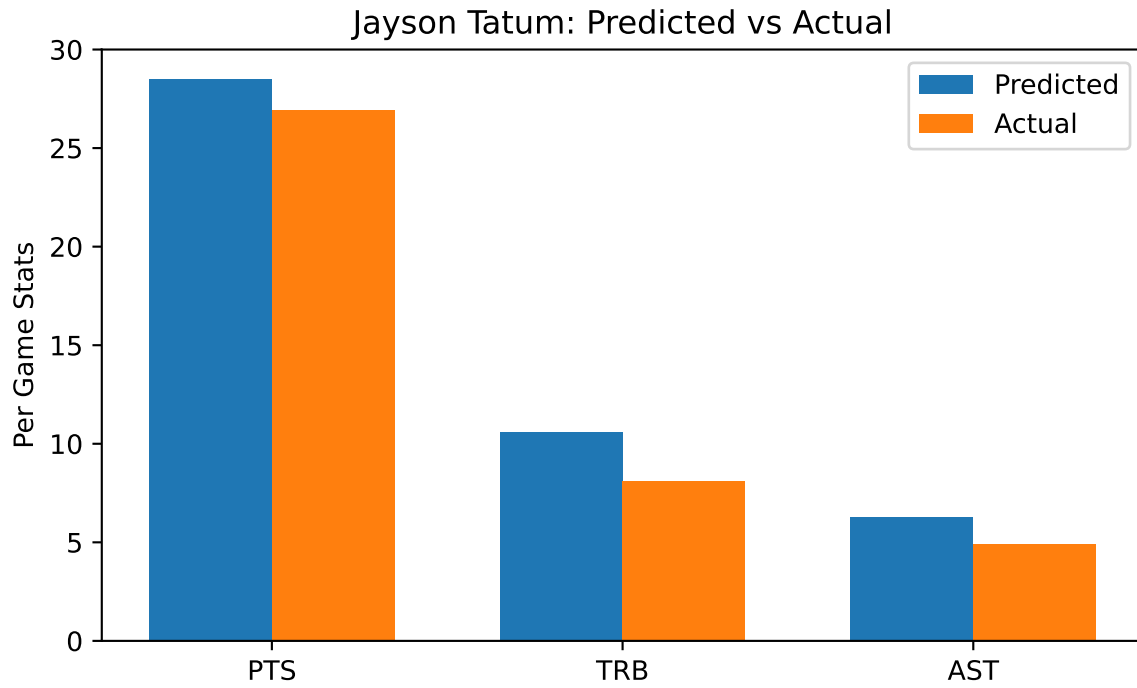
```
warnings.warn(  
/Users/vinniecarbone/Documents/3255/Presentation/tfenv/lib/python3.11/site-  
packages/sklearn/utils/validation.py:2691: UserWarning: X does not have valid feature names,  
warnings.warn(  
/Users/vinniecarbone/Documents/3255/Presentation/tfenv/lib/python3.11/site-  
packages/sklearn/utils/validation.py:2691: UserWarning: X does not have valid feature names,  
warnings.warn(  
  
/Users/vinniecarbone/Documents/3255/Presentation/tfenv/lib/python3.11/site-  
packages/sklearn/utils/validation.py:2691: UserWarning: X does not have valid feature names,  
warnings.warn(  

```

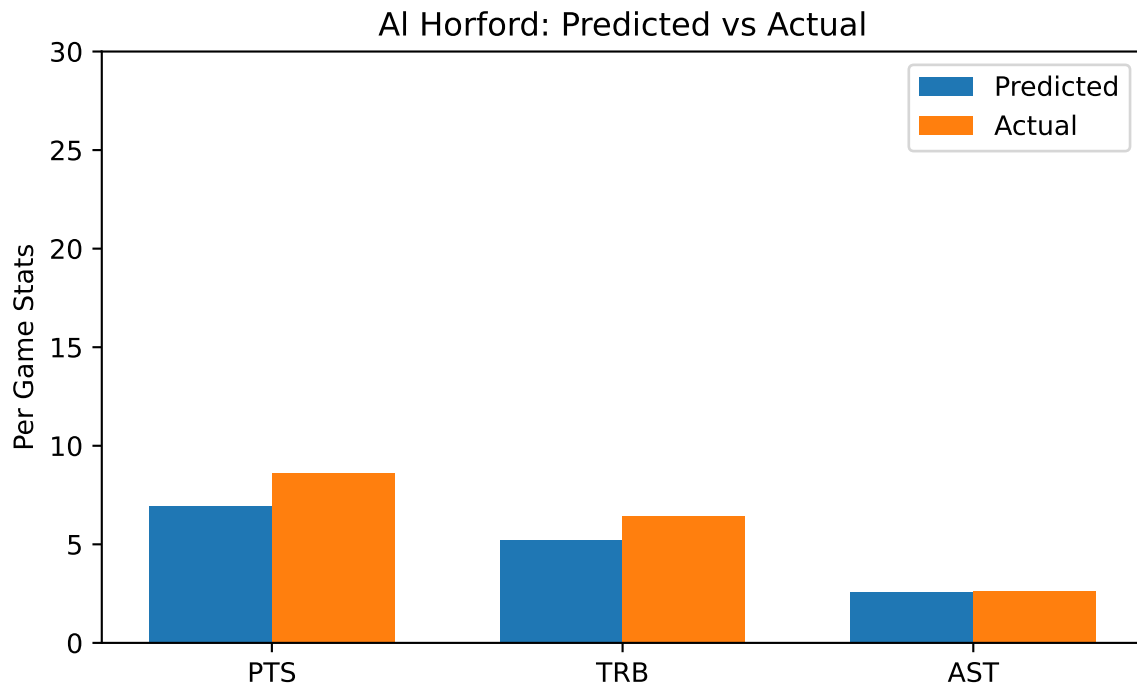


```
/Users/vinniecarbone/Documents/3255/Presentation/tfenv/lib/python3.11/site-  
packages/sklearn/utils/validation.py:2691: UserWarning: X does not have valid feature names,  
warnings.warn(  

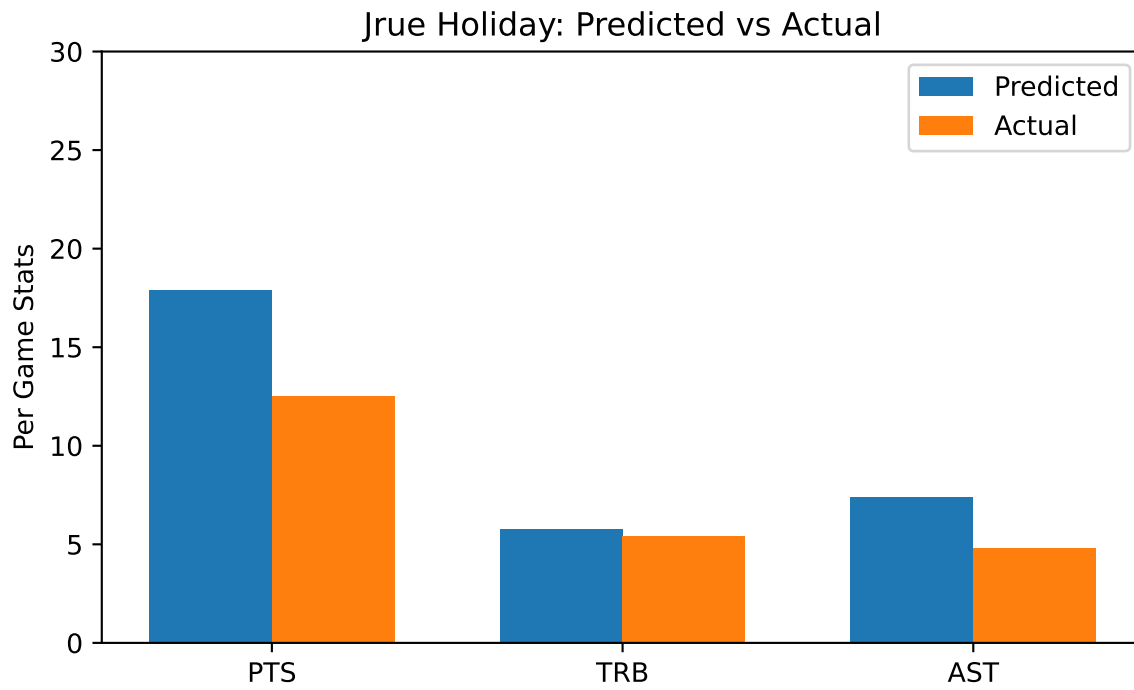
```



```
/Users/vinniecarbone/Documents/3255/Presentation/tfenv/lib/python3.11/site-  
packages/sklearn/utils/validation.py:2691: UserWarning: X does not have valid feature names,  
warnings.warn(
```

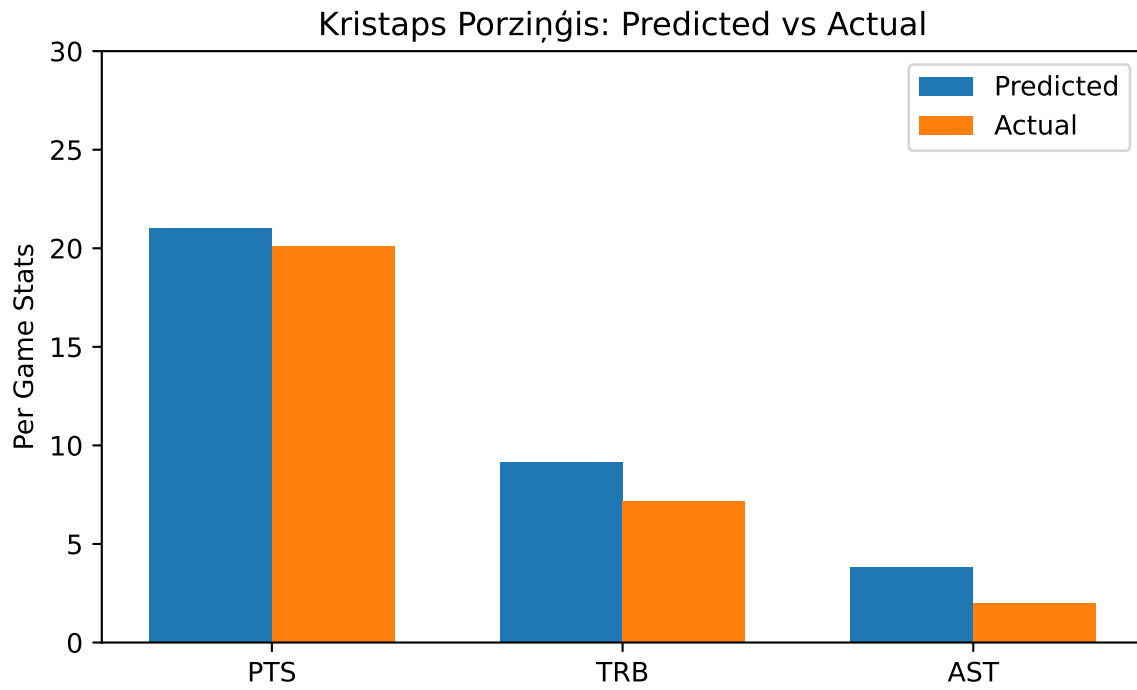


```
/Users/vinniecarbone/Documents/3255/Presentation/tfenv/lib/python3.11/site-  
packages/sklearn/utils/validation.py:2691: UserWarning: X does not have valid feature names,  
warnings.warn(
```

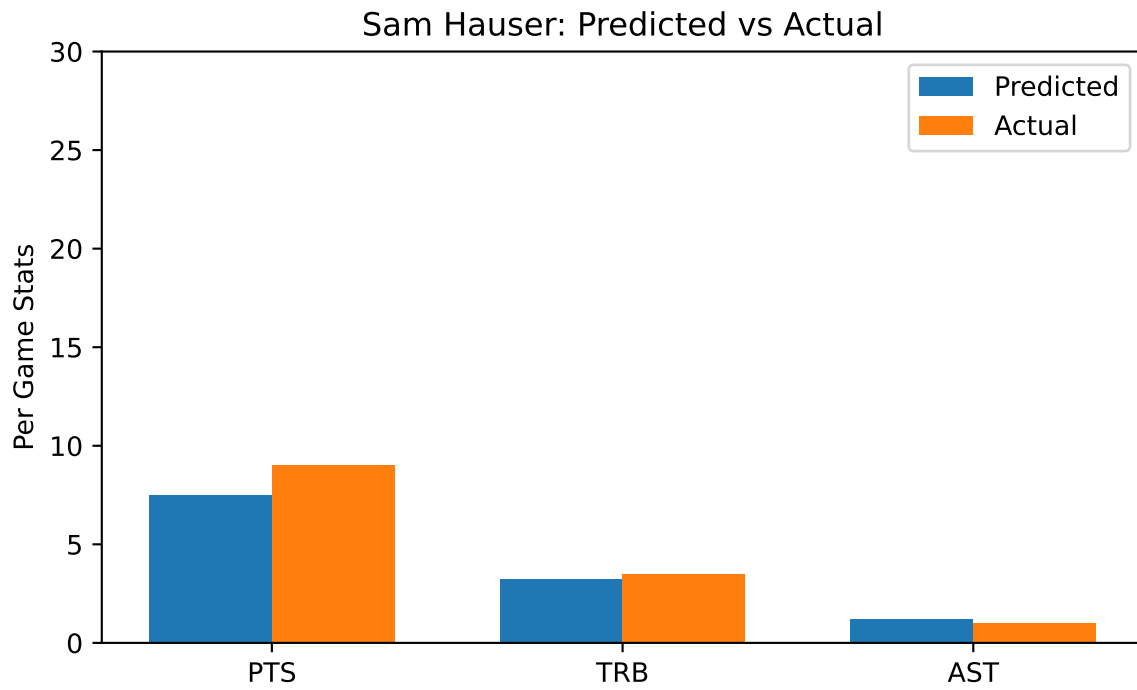


```
/Users/vinniecarbone/Documents/3255/Presentation/tfenv/lib/python3.11/site-  
packages/sklearn/utils/validation.py:2691: UserWarning: X does not have valid feature names,  
warnings.warn(
```

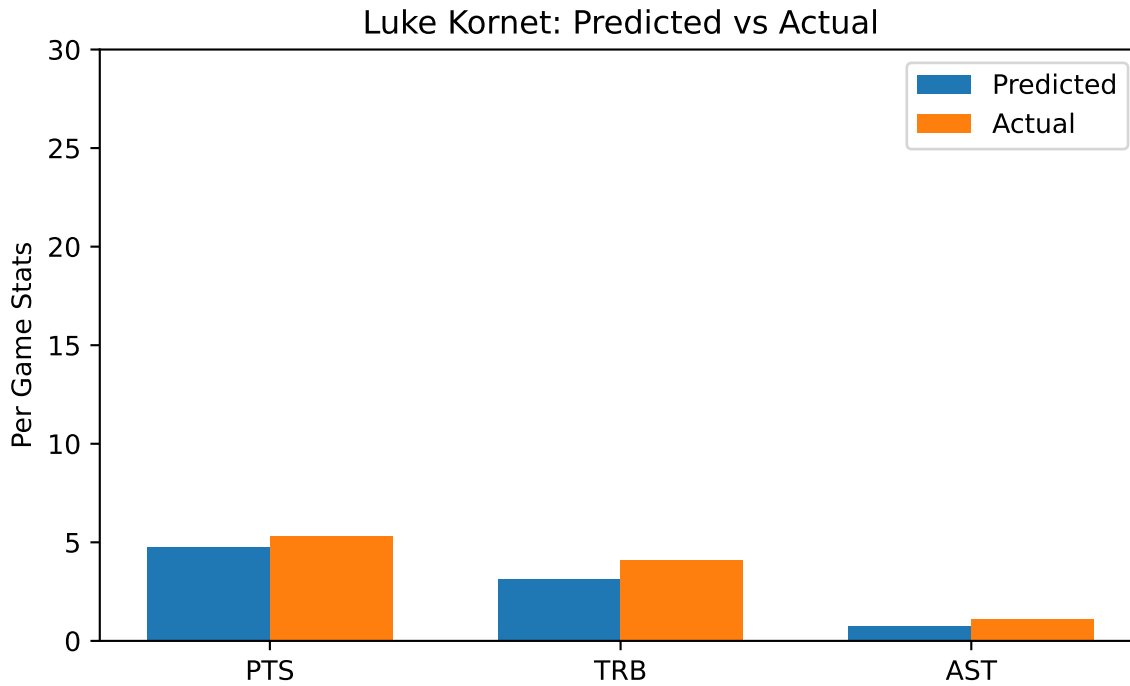




```
/Users/vinniecarbone/Documents/3255/Presentation/tfenv/lib/python3.11/site-  
packages/sklearn/utils/validation.py:2691: UserWarning: X does not have valid feature names,  
warnings.warn(
```



```
/Users/vinniecarbone/Documents/3255/Presentation/tfenv/lib/python3.11/site-  
packages/sklearn/utils/validation.py:2691: UserWarning: X does not have valid feature names,  
warnings.warn(
```



## Limitations of This Specific Model

- Only used basic statistics:
  - Age, minutes, shooting, previous stats, etc
  - Does NOT include: injuries, team changes, role changes, etc
- Dataset is relatively small
  - Fewer examples can limit how well the model learns
- Simple neural network (64 → 32 layers)
  - More complex models could improve performance
- Does not account for:
  - Team system
  - Teammates
  - Coaching changes

## Limitations of Neural Networks in General

- Neural networks require **large amounts of data**
  - Performance suffers with limited or noisy data
- Often difficult to interpret
  - Difficult to understand how predictions are made
- Training can be:
  - Time-consuming
  - Resource-intensive

## Further Reading

- [Other applications of neural networks] (<https://www.ibm.com/think/topics/neural-networks>)
- <https://course.elementsofai.com/en-ie/5/1>
- [Creating your own neural network in python] (<https://medium.com/@soukaina./creating-your-first-neural-network-model-in-python-a-detailed-guide-681ce456b90e>)